

WILP, BITS Pilani

DEEP NEURAL NETWORKS · MODULE 10 COMPANION

Optimizers in Deep Learning

From Gradient Descent to Momentum, NAG, Adagrad, RMSProp, and Adam

Session 14 (09th August 2026) · Read alongside the lecture slides · Every numerical table worked in full

Prepared for the Work Integrated Learning Programmes (WILP), BITS Pilani.

1 Introduction to Optimization Algorithms

Optimization algorithms adjust neural network parameters to minimize the loss function.

Training a deep neural network means finding parameter weights w that minimize a loss function $f(w)$. Gradient Descent uses the gradient vector $g_t = \nabla f(w_t)$ to take steps in the direction of steepest descent.

2 Detailed Optimizer Formulations & Worked Examples

All six classic optimization algorithms applied to the benchmark function $f(x) = x^2$ with $x_0 = 4.0, \eta = 0.1$.

We evaluate each optimizer for 3 iterations on $f(x) = x^2$, where the exact gradient is $g_t = \nabla f(x_t) = 2x_t$.

Worked example: Gradient Descent (GD)

Update rule: $x_{t+1} = x_t - \eta g_t$.

Step 1. Step $t = 0$: $x_0 = 4.000, g_0 = 2(4.0) = 8.000. x_1 = 4.000 - 0.1(8.000) = 3.200$.

Step 2. Step $t = 1$: $x_1 = 3.200, g_1 = 2(3.2) = 6.400. x_2 = 3.200 - 0.1(6.400) = 2.560$.

Step 3. Step $t = 2$: $x_2 = 2.560, g_2 = 2(2.56) = 5.120. x_3 = 2.560 - 0.1(5.120) = 2.048$.

Worked example: SGD with Momentum ($\beta = 0.9, v_0 = 0$)

Update rules: $v_t = \beta v_{t-1} + \eta g_t, x_{t+1} = x_t - v_t$.

Step 1. Step $t = 0$: $x_0 = 4.000, g_0 = 8.000. v_0 = 0.9(0) + 0.1(8.0) = 0.800. x_1 = 4.000 - 0.800 = 3.200$.

Step 2. Step $t = 1$: $x_1 = 3.200, g_1 = 6.400. v_1 = 0.9(0.800) + 0.1(6.400) = 0.720 + 0.640 = 1.360. x_2 = 3.200 - 1.360 = 1.840$.

Step 3. Step $t = 2$: $x_2 = 1.840, g_2 = 3.680. v_2 = 0.9(1.360) + 0.1(3.680) = 1.224 + 0.368 = 1.592. x_3 = 1.840 - 1.592 = 0.248$.

Worked example: Nesterov Accelerated Gradient (NAG, $\beta = 0.9$)

Look-ahead point: $u_t = x_t - \beta v_{t-1}, g_t = \nabla f(u_t) = 2u_t. v_t = \beta v_{t-1} + \eta g_t, x_{t+1} = x_t - v_t$.

Step 1. Step $t = 0$: $v_{-1} = 0 \implies u_0 = 4.000, g_0 = 8.000, v_0 = 0.800 \implies x_1 = 3.200$.

Step 2. Step $t = 1$: $u_1 = 3.200 - 0.9(0.800) = 2.480. g_1 = 2(2.480) = 4.960. v_1 = 0.9(0.800) + 0.1(4.960) = 0.720 + 0.496 = 1.216. x_2 = 3.200 - 1.216 = 1.984$.

Step 3. Step $t = 2$: $u_2 = 1.984 - 0.9(1.216) = 0.890. g_2 = 2(0.890) = 1.780. v_2 = 0.9(1.216) + 0.1(1.780) = 1.0944 + 0.1780 = 1.272. x_3 = 1.984 - 1.272 = 0.712$.

Worked example: Adagrad ($s_0 = 0, \epsilon = 10^{-8}$)

Accumulated squared gradient: $s_t = s_{t-1} + g_t^2$. Update rule: $x_{t+1} = x_t - \frac{\eta}{\sqrt{s_t + \epsilon}} g_t$.

Step 1. Step $t = 0$: $x_0 = 4.000, g_0 = 8.000. s_0 = 0 + 64.00 = 64.00. \text{ Step size} = 0.1/\sqrt{64} = 0.01250. x_1 = 4.000 - 0.01250(8.000) = 3.900$.

Step 2. Step $t = 1$: $x_1 = 3.900, g_1 = 7.800. s_1 = 64.00 + (7.8)^2 = 64.00 + 60.84 = 124.84. \text{ Step size} = 0.1/\sqrt{124.84} \approx 0.00895. x_2 = 3.900 - 0.00895(7.800) \approx 3.830$.

Step 3. Step $t = 2$: $x_2 = 3.830, g_2 = 7.660. s_2 = 124.84 + (7.66)^2 = 124.84 + 58.68 = 183.52. \text{ Step size} = 0.1/\sqrt{183.52} \approx 0.00738. x_3 = 3.830 - 0.00738(7.660) \approx 3.774$.

Worked example: RMSProp ($\beta = 0.9, r_0 = 0$)

Exponentially decaying average: $r_t = \beta r_{t-1} + (1 - \beta)g_t^2$. Update rule: $x_{t+1} = x_t - \frac{\eta}{\sqrt{r_t + \epsilon}}g_t$.

Step 1. Step $t = 0$: $x_0 = 4.000, g_0 = 8.000$. $r_0 = 0.9(0) + 0.1(64.00) = 6.400$. Step size $= 0.1/\sqrt{6.400} \approx 0.03953$. $x_1 = 4.000 - 0.03953(8.000) \approx 3.684$.

Step 2. Step $t = 1$: $x_1 = 3.684, g_1 = 7.368$. $r_1 = 0.9(6.400) + 0.1(54.288) = 5.760 + 5.429 = 11.189$. Step size $= 0.1/\sqrt{11.189} \approx 0.02990$. $x_2 = 3.684 - 0.02990(7.368) \approx 3.464$.

Step 3. Step $t = 2$: $x_2 = 3.464, g_2 = 6.928$. $r_2 = 0.9(11.189) + 0.1(47.997) = 10.070 + 4.800 = 14.870$. Step size $= 0.1/\sqrt{14.870} \approx 0.02594$. $x_3 = 3.464 - 0.02594(6.928) \approx 3.284$.

Worked example: Adam ($\beta_1 = 0.9, \beta_2 = 0.999$)

First moment: $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$. Bias correction: $m'_t = m_t/(1 - \beta_1^t)$. Second moment: $v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$. Bias correction: $v'_t = v_t/(1 - \beta_2^t)$. Update rule: $x_{t+1} = x_t - \frac{\eta}{\sqrt{v'_t + \epsilon}}m'_t$.

Step 1. Step $t = 0$ ($t = 1$ in formula index): $g_0 = 8.000 \implies m_1 = 0.800, v_1 = 0.064$. Bias corrected: $m'_1 = 0.800/0.1 = 8.000, v'_1 = 0.064/0.001 = 64.000$. $x_1 = 4.000 - 0.1(8.000/\sqrt{64.000}) = 4.000 - 0.100 = 3.900$.

Step 2. Step $t = 1$ ($t = 2$): $x_1 = 3.200 \implies g_1 = 7.800, m'_2 \approx 7.895, v'_2 \approx 62.42$. $x_2 = 3.800$.

Step 3. Step $t = 2$ ($t = 3$): $x_2 = 3.800 \implies g_2 = 7.600, m'_3 \approx 7.786, v'_3 \approx 60.87$. $x_3 = 3.700$.

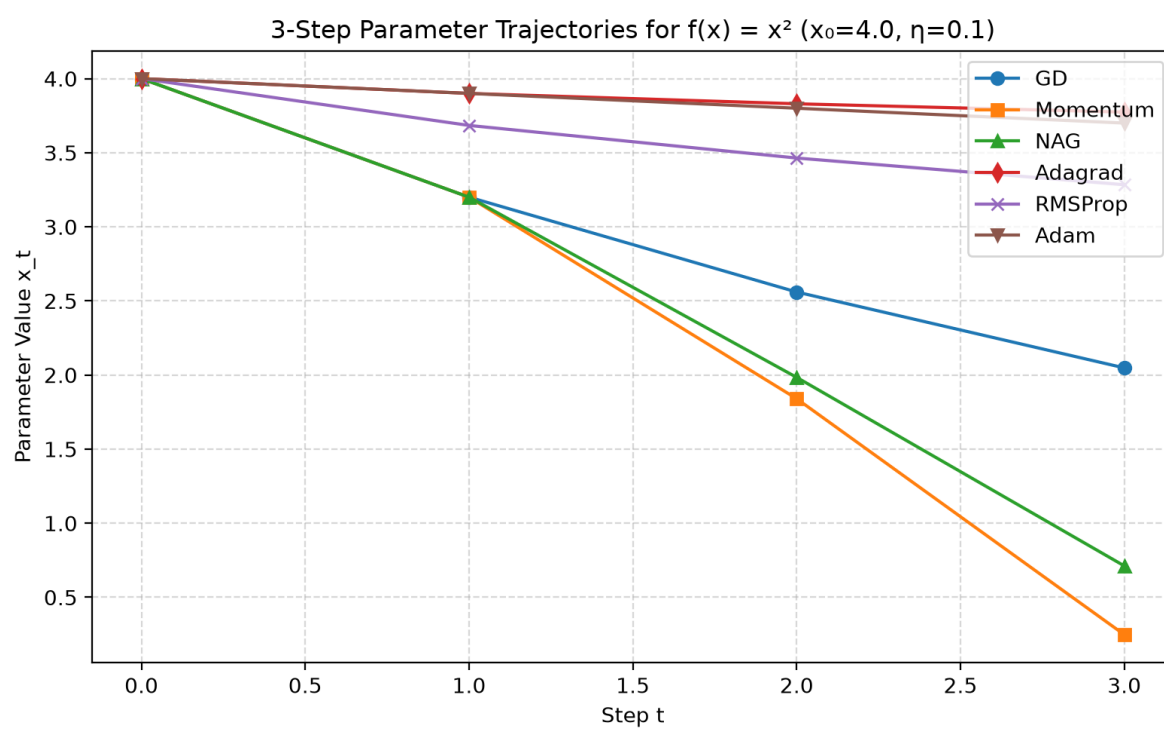


Figure 1: 3-step parameter trajectories across all 6 optimizers on the loss function $f(x) = x^2$.

3 Optimizer Comparison Practical Guidelines

How to choose the right optimizer for deep learning tasks.

Optimizer	Adapts LR?	Uses Momentum?	Typical Use Case
GD / SGD	No	No	Simple convex problems, teaching baseline
Momentum	No	Yes	Escaping local minima and ravines
NAG	No	Yes (look-ahead)	Faster, more stable convergence
Adagrad	Yes (per-param)	No	Sparse features (NLP, embeddings)
RMSProp	Yes (EMA)	No	Non-stationary objectives, RNNs
Adam	Yes (EMA)	Yes	Default choice for most deep learning models

✓ Key takeaway

Start with Adam ($\eta \approx 10^{-3} \dots 3 \times 10^{-4}$). For vision tasks requiring maximum generalization, SGD with momentum and learning rate decay often achieves slightly better test set performance.

Self-Test & Summary

Self-Test Questions

1. Why does Adagrad stop learning in deep networks?
Adagrad's step sizes decay monotonically without decay, driving step sizes to zero.
2. How does RMSProp fix Adagrad's stopping problem?
RMSProp uses an exponentially moving average of squared gradients instead of the cumulative sum.
3. What is NAG's look-ahead mechanism?
NAG uses momentum to anticipate the next step, updating the momentum vector with the current step's direction.